Homework10

Task 1 (1 points):

Open your GitHub repository in Codespace.

1. In the terminal run the command: `pip install -r requirements.txt`
2. In the terminal run the command: `cd HW/Homework10/University/`
3. In the terminal run the command: `cp .env.example .env`
4. In the **.env** file, paste the copied connection string from task 1 after: **DB_CONNECTION_STRING=**
5. You'll now need to parse the connection string to get the **DB_HOST** and **DB_PASSWORD** values. Example:

```
postgresql://neondb_owner:password@ep-example-123456-pooler.us-east-1.aws.neon.tech/university?sslmode=require&channel_binding=require
```

6. The **host** can be found between **@** and the first **/** after **@**. The **password** can be found between the first **:** before **@** and **@** in the connection string.

```
DB_HOST=ep-example-123456-pooler.us-east-1.aws.neon.tech
DB_PASSWORD=password
```

Task 2 (9 points):

Create a file called **Database.py** inside **HW/Homework10/University/** and at the top of the file, import the following:

```
psycopg2
sql from psycopg2
os
tabulate from tabulate
load_dotenv from dotenv
```

Create a class named **Database** with the following methods:

1. `def __init__(self, ddl_file: str ="ddl.sql", data_dir="DATA") -> None:`
   Define the following variables:
   self.connection assigned `None`
   self.ddl_file assigned `ddl_file`
   self.data_dir assigned `data_dir`
   self.relations assigned `[]`
2. `def connect(self, username: str = None, password: str = None) -> None:`
   Connect to the database using the **connect_to_db()** function from setup.py as a guide. Instead of returning the connection/None, assign it to self.connection. For the **user** variable assignment, provide a ternary expression that assigns `os.getenv("DB_USER")` to **user** if `username` is None. Otherwise it assigns `username` to **user**. (This should be a one-liner.) For the **password** variable assignment, provide a ternary expression that assigns `os.getenv("DB_PASSWORD")` to **password** if `password` is None. Otherwise it assigns `password` to **password**. (This should be a one-liner.)
3. `def close(self) -> None:`
   If the self.connection is not None, close self.connection and assign it None. Then assign an empty list to self.relations.
4. `def __enter__(self) -> "Database":`
   Call self.connect() and return self.
5. `def __exit__(self, exc_type, exc_val, exc_tb) -> None:`
   Call self.close().
6. `def get_relations(self) -> list[str]:`
   Define this method using the get_relation_names() function from load.py. Use self.connection and self.ddl_file.
7. `def create_tables(self) -> None:`
   Define this method using the create_tables() function from setup.py. Use self.connection and self.ddl_file. At the end of the try block, assign self.relations the return of self.get_relations().

8. `def load_data(self) -> None:`
   Define this method using the load_data() function from load.py. Use self.connection and self.data_dir.
9. `def setup(self) -> None:`
   Call self.create_tables() and self.load_data().
10. `def execute_query(self, query) -> None:`
    Define this method using the execute_query() function from query.py. Use self.connection.
11. `def show_tables(self) -> None:`
    Define this method using the show_tables() function from load.py. Use self.connection. (Loop through self.get_relations())
12. `def drop_tables(self) -> None:`
    Define this method using the drop_tables() function from drop.py. Use self.connection. At the end of the try block, assign an empty list to self.relations.
13. `def version(self) -> None:`
    Define this method using the version() function from test.py. Use self.connection.
14. `def reset_tables(self) -> None:`
    Call the **drop_tables()**, **create_tables()**, and **load_data()** methods in that order.
15. `def factory_reset(self) -> None:`
    Define this method using the factory_reset() function from reset.py. At the beginning of the try block, check if self.connection is not None. If it is, close the connection. After the if statement in the finally block, call the connect() method then the setup() method.
16. `def show_schema(self) -> None:`
    Define this method using the show_schema() function from load.py as a guide. Use self.connection and loop through self.get_relations().
17.
```
@staticmethod
def create_database(db_name: str) -> None:
```
    Define this static method using the create_database() function from setup.py as a guide. They should be identical.
18.
```
@staticmethod
def drop_database(db_name: str) -> None:
```
    Define this static method using the drop_database() function from setup.py as a guide. They should be identical.

Push your work to GitHub using the following commands:

```
git pull
git add .
git –m “homework10 upload”
git push
```